Predicting Hits with KNN: 2026 Balls in Play

Goal

The goal of this project was to build on my earlier SVM project (LA_EV 2026 SVM Model), which predicted whether a ball in play was a hit or an out using only launch angle and exit velocity. This time I used a second classification method from ISYE 6501, k nearest neighbors (KNN), through the kknn package in R. I followed the same general approach as the SVM project, but added more predictors to see if giving the model more information about each batted ball and swing would change anything. The nine predictors were launch speed (exit velocity), launch angle, hit distance, ballpark, bat speed, swing length, attack angle, attack direction, and swing path tilt.

Methodology

1. A CSV containing every batted ball from the 2026 regular season (March through September) was downloaded from Baseball Savant and read into R using read.csv with header = TRUE. The kknn library was loaded to use the train.kknn and kknn functions.
2. The response variable, hit, was created.
    a. Using the events column, a ball in play was marked as a hit if the outcome was a single, double, triple, or home run.
    b. ifelse was used to convert that TRUE/FALSE column into 1 for a hit and 0 for an out.
3. The data was cleaned so that only balls in play with all nine predictors were included.
    a. A vector called vars listed the nine predictor columns plus hit. Ballpark was represented by the home_team column, since every game is played in the home team's park.
    b. complete.cases() kept only the rows with no missing values in those columns, and only those columns were kept and saved as bip.
    c. home_team was converted with as.factor() so it would be treated as a category instead of text.
    d. table() and nrow() were run on bip to confirm the row count and find the baseline, which is the accuracy a model would get by predicting an out every single time.
4. hit was converted with as.factor() so kknn would treat this as a classification problem (hit or out) instead of trying to predict a number.
5. The data was split into a training set and a test set.
    a. set.seed(42) was placed right before sample() so the same split would come out every time the script was rerun.
    b. sample(nrow(bip), round(0.75 * nrow(bip))) picked 75% of the row numbers at random, which were saved as idx and used to pull those rows out of bip as bip_train.

    c. bip[-idx,] grabbed every remaining row and saved it as bip_test.
    d. An earlier version of this project trained on a 10,000 ball sample, the same as the SVM project. SVMs slow down a lot as the data gets bigger, but KNN doesn't really have a training step, and more training rows give it more neighbors to compare against, so the split was changed to 75/25.
6. train.kknn was used to run leave one out cross validation on bip_train to find a good value of k.
    a. kernel = "rectangular" was used so every neighbor gets an equal vote, and scale = TRUE put all the predictors on the same scale so no single variable would dominate the distance.
    b. Only odd values of k were tested so there could never be a tied vote.
    c. A wide search was run first (1 to 301 in steps of 50). Once the best values were found to be between 151 and 251, a narrower search tested every odd value in that range.
    d. 1 minus MISCLASS was printed to get the cross validated accuracy for each k.
7. The final model was built and tested.
    a. k_best was set to 191, the best value from the narrow search.
    b. kknn was run with bip_train as the training data and bip_test as the test data, using the same kernel and scale settings. Each test ball was classified by a majority vote of its 191 nearest neighbors in the training set.
    c. The predictions were pulled from fitted.values and saved as pred_test.
    d. Test accuracy was calculated, and table() was run comparing pred_test to the actual hit column of bip_test to get a confusion matrix.
8. Exit velocity was plotted against launch angle for the test set twice, once colored by the actual outcome and once by the predicted outcome, with red for hits and gray for outs.
9. A third plot was made to show where the model went wrong.
    a. A nested ifelse labeled each test ball as Correct, Missed hit (actually a hit but predicted an out), or False hit (actually an out but predicted a hit).
    b. table() was run on those labels to make sure they matched the confusion matrix.
    c. points() was used to layer the plot, with correct predictions drawn first in light gray and the two kinds of errors drawn on top in blue and orange.
10. One more cross validation run tested k from 1 to 301 in steps of 10, and the results were saved with write.csv so the slow search wouldn't need to be rerun. Accuracy was then plotted against k, with reference lines at the baseline and at k = 191.

Results

Baseline. After cleaning, 116,651 balls in play had all nine predictors. Of those, 78,988 were outs, or about 67.7%, and 37,663 were hits. That means a model that just predicts an out every time would already be right about 67.7% of the time, so any model has to beat that to be worth anything.

```
> # Baseline: share of hits and outs in the cleaned data
> print(table(bip$hit))

    0     1
78988 37663
> print(nrow(bip))
[1] 116651
```

Choosing k. The wide search showed accuracy climbing fast at first and then leveling off. k = 1 was the worst by a wide margin, which makes sense since every ball just copies whatever happened to its single closest neighbor. The best value in the wide search was k = 201, so the narrow search focused on 151 to 251.

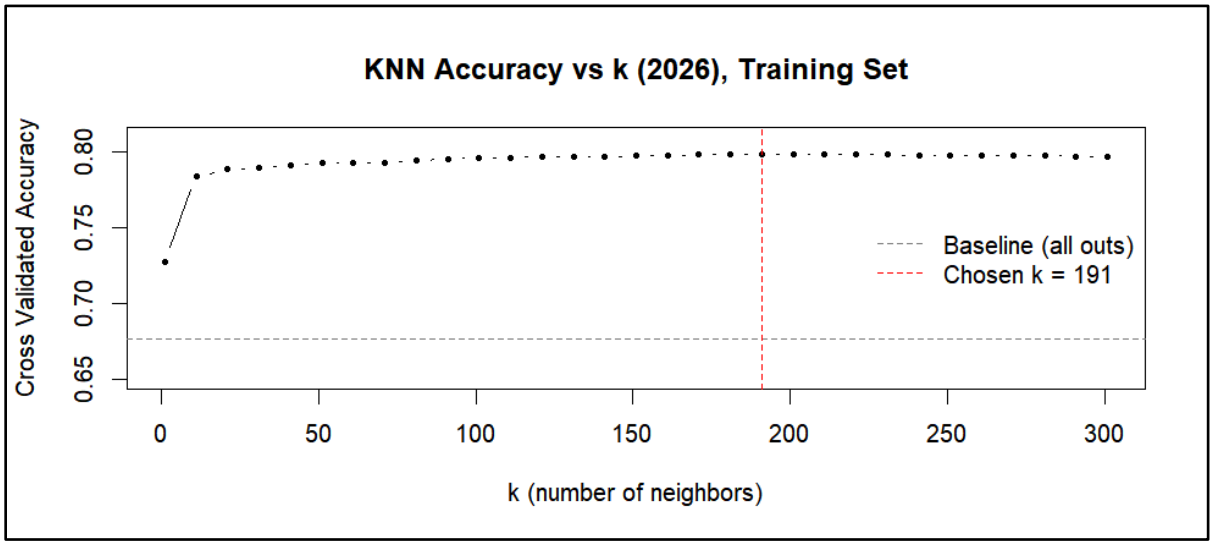
Cross validated accuracy from the wide search:

k	Cross validated accuracy
1	0.7279
51	0.7927
101	0.7962
151	0.7975
201	0.7983
251	0.7978
301	0.7972

In the narrow search, k = 191 had the highest accuracy at 0.7987. Every value from about 171 to 223 landed between 0.7982 and 0.7987, so the top of the curve is very flat. The results of the narrow search are shown below.

```
> # Cross validated accuracy for this k (1 minus the misclassification rate)
> print(1 - model_cv$MISCLASS)
    rectangular
151   0.7975037
153   0.7974694
155   0.7971836
157   0.7974008
159   0.7973665
161   0.7974808
163   0.7976980
165   0.7977323
167   0.7980637
169   0.7979952
171   0.7984524
173   0.7985781
175   0.7982009
177   0.7981895
179   0.7982009
181   0.7983723
183   0.7982238
185   0.7984295
187   0.7986352
189   0.7986352
191   0.7987267
193   0.7985781
195   0.7985895
197   0.7981552
199   0.7982809
201   0.7982695
203   0.7982580
205   0.7985781
207   0.7984524
209   0.7982238
211   0.7983838
213   0.7983495
215   0.7982123
217   0.7983723
219   0.7985781
221   0.7984752
223   0.7983838
225   0.7981895
227   0.7980752
229   0.7983838
231   0.7983381
233   0.7979151
235   0.7981437
237   0.7982695
239   0.7981780
241   0.7977551
243   0.7977666
245   0.7979380
247   0.7978694
249   0.7977666
251   0.7978123
```

The plot below shows cross validated accuracy for k from 1 to 301 in steps of 10, with the baseline and the chosen k marked.



Test set. With k = 191, the model's test accuracy was 0.7991 on 29,163 balls in play it had never seen, almost the same as the 0.7987 cross validated accuracy on the training set.

```
> # Save the predicted class (0 or 1) for each test ball
> # (kknn already made these when the model was built with test = bip_test)
> pred_test <- model$fitted.values
>
> # Test accuracy: share of test balls classified correctly
> print(sum(pred_test == bip_test$hit) / nrow(bip_test))
[1] 0.7990605
>
> # Confusion matrix (rows = predicted, columns = actual)
> print(table(pred_test, bip_test$hit))

pred_test     0     1
        0 18517  4582
        1  1278  4786
>
```

Confusion matrix results on the test set (29,163 balls in play):

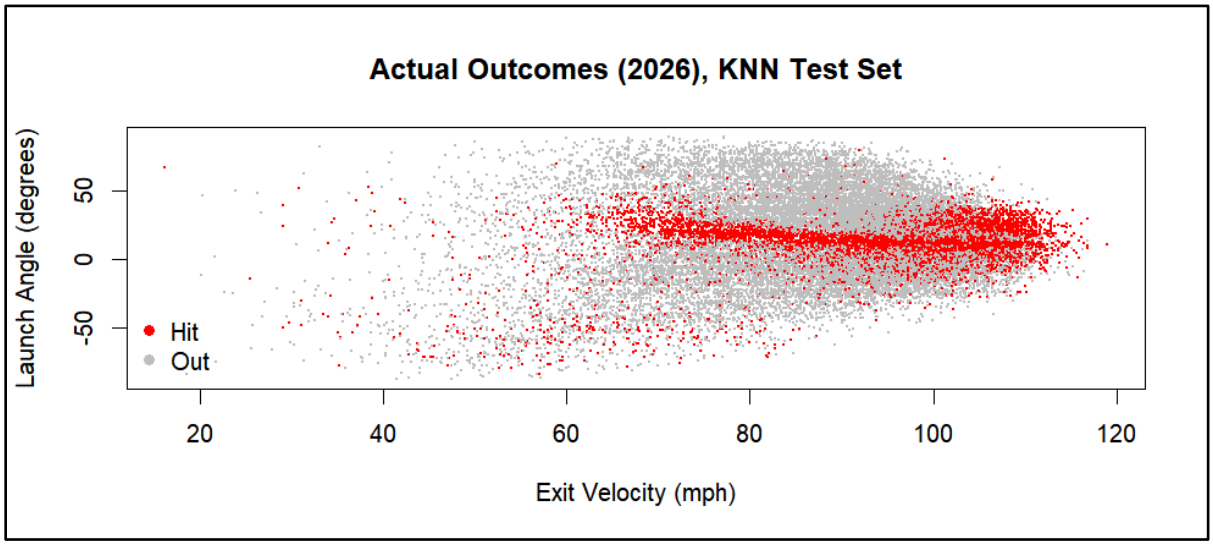
Correct outs	Missed hits	False hits	Correct hits
18,517	4,582	1,278	4,786

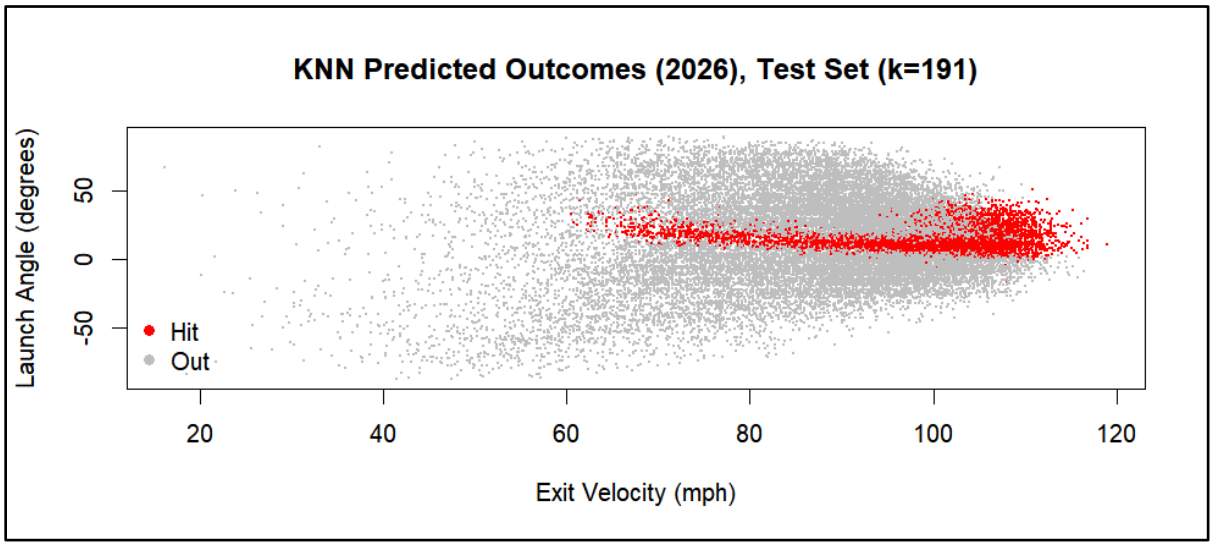
The same confusion matrix as rates:

Hits caught	Right when it predicts a hit	Outs correct
51%	79%	94%

Actual vs predicted outcomes. The model predicts hits in two places: a band of line drives between about 10 and 25 degrees across most exit velocities, and a block of

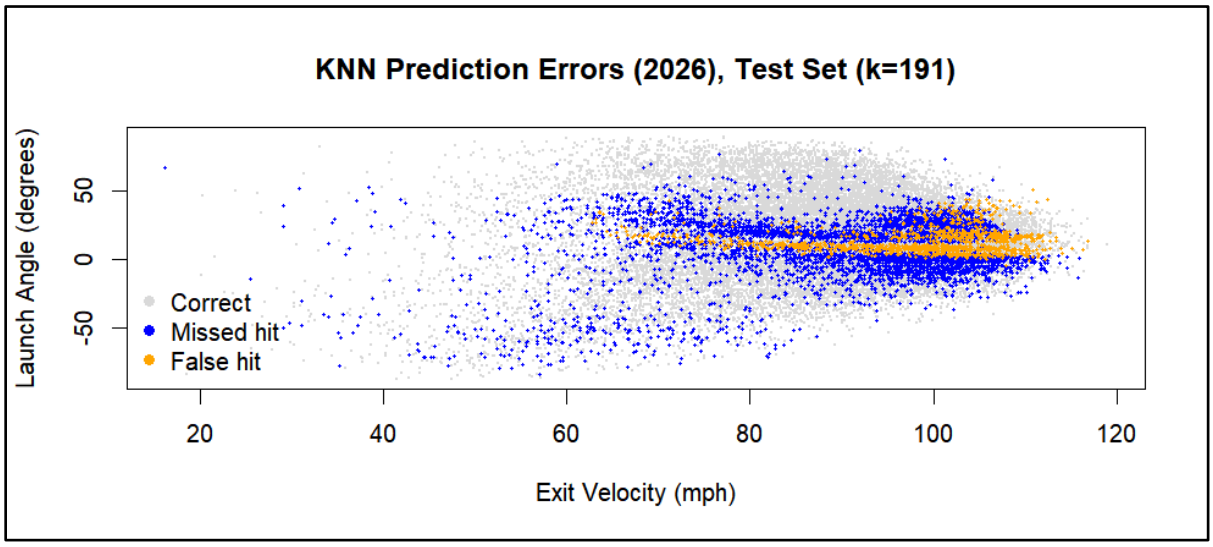
hard hit balls above 100 mph between about 10 and 45 degrees. Everything else is predicted as an out.





Prediction errors. Labeling each test ball gave 23,303 correct, 4,582 missed hits, and 1,278 false hits, which matches the confusion matrix. The plot below shows where each type of error falls.

```
> # ---- Plot: where the model gets it wrong ----
> # Label each test ball by how its prediction compares to what actually happened
> #   Correct    = prediction matched the actual outcome
> #   Missed hit = actually a hit, but the model called it an out
> #   False hit  = actually an out, but the model called it a hit
> result <- ifelse(pred_test == bip_test$hit, "Correct",
+                  ifelse(bip_test$hit == 1, "Missed hit", "False hit"))
>
> # Count of each label (should line up with the confusion matrix)
> print(table(result))
result
   Correct  False hit Missed hit
     23303       1278       4582
```



Discussion

The biggest takeaway is that adding seven more predictors didn't really help. The KNN model reached 0.7991 test accuracy with nine predictors, while the SVM reached 0.8064 using only launch angle and exit velocity. The two aren't a perfect comparison. The KNN data has about 4,200 fewer balls in play since it only kept balls with all nine predictors, the baseline moved from 67.5% to 67.7%, and the two models were trained on very different amounts of data. Still, the extra predictors didn't seem to add much, and that's even with hit distance in the model, which might partly give away the answer (more on that below). My understanding is that KNN gives every scaled predictor an equal say when it measures how close two balls are, so a predictor that doesn't tell you much about whether a ball falls in still counts just as much as exit velocity. That can pull in neighbors that are similar in ways that don't matter.

The model also isn't overfitting. Cross validated accuracy on the training set (0.7987) and accuracy on the test set (0.7991) were almost identical, so whatever the model learned held up on balls it had never seen.

Choosing k ended up mattering less than I expected once it got reasonably large. Most of the improvement came between k = 1 and k = 11, and past about 150 the curve is basically flat. k = 191 had the best accuracy, but everything from 171 to 223 was within

about 0.0006 of it, which is only about 50 balls out of 87,488. So 191 is really the middle of a plateau rather than a sharp peak. The size of the training set also changed the answer. With the earlier 10,000 ball sample, the best k was 49 at 0.790. With 87,488 balls, it moved to 191. More data packs similar balls closer together, so the model can afford to poll more neighbors.

Most of the model's mistakes make sense from a baseball standpoint. The missed hits are mostly ground balls that found a hole, anywhere from about 50 mph up to hard hit grounders around 105 mph, along with soft contact under 60 mph and bloops and flares just above the line drive band. The false hits are most likely line drives hit right at a fielder and deep fly balls caught near the wall. That also explains why the model gets 94% of outs right but only catches 51% of hits. Because it goes by majority vote, a type of ball that turns into a hit less than half the time gets called an out every time, even if a good share of them fall in. Whether those balls drop depends on things none of the nine predictors capture, like where the ball was hit, where the fielders were standing, and how fast the runner is.

There are a couple of limitations worth mentioning. Hit distance might be partly giving away the answer. For ground balls, I suspect the distance reflects where the ball ended up, and a grounder that gets through the infield will naturally travel farther than one that gets fielded. Using home_team for ballpark is also a little off for the handful of games played at neutral sites each season.

For next steps, I would use a random forest to rank the nine predictors, building on the variable importance work from ISYE 6501, and then drop the weakest ones from the KNN model to see if accuracy holds or even improves. That would also show how much of the work hit distance is doing. I would also add spray angle, calculated from the hit coordinates and flipped for left and right handed batters, since that covers a lot of what the model is missing. Finally, I would try lowering the voting cutoff, for example calling a hit when 40% of the neighbors were hits, to catch more of the missed hits at the cost of some false ones.